RevisionBench: A Reproducible Failed-Construction Case Study for Abstract-Local Scientific Claim Updates

Anonymous Authors

Abstract

Recent work on evolving knowledge evaluates world knowledge, temporal QA, stale retrieval, and temporal fact verification, but not whether a system can reject an obsolete scientific claim from an earlier paper version using only the latest abstract as local evidence. RevisionBench was intended to fill that narrow gap. This paper reports the implemented workspace as a resource-audit note rather than a benchmark paper because the required human validation never finished. Starting from 180 raw multi-version arXiv papers, deterministic filtering retains 135 in-scope records across `cs.AI`, `cs.CL`, `cs.CV`, and `cs.LG`. The executed mining pipeline yields 833 triggered candidate claim-update pairs, 28 semantic-audit candidates, 1,428 rejected candidates, and a stratified pilot export of 118 items for each seed in {17, 23, 42}. However, the two-annotator plus adjudication workflow was not completed, so the final adjudicated benchmark size is 0, dev and test splits are both 0, and all benchmark-dependent baselines are skipped. The contribution of this paper is therefore transparent artifact accounting: it documents a reproducible candidate-mining pipeline, the exact stage at which benchmark construction failed, and the limits of what can honestly be claimed from the current workspace.

1 Introduction

Benchmarks for changing knowledge have advanced quickly in 2024–2025, but the dominant settings still emphasize world knowledge, temporal QA, or outdated retrieval rather than revision-aware scientific claims. EvoWiki, EvolveBench, HoH, ChronoFact, and evolveQA all probe different aspects of evolving information [Tang et al., 2025, Ouyang et al., 2025, Zhu et al., 2025, Barik et al., 2025, Nakshatri et al., 2025]. Scientific revision corpora also already exist: arXivEdits aligns scientific revisions, CASIMIR scales revision alignment across OpenReview, and VitaminC shows that document revisions can be converted into verification examples [Jiang et al., 2022, Jourdan et al., 2024, Schuster et al., 2021]. The remaining defensible gap is narrower: there is still no locally evidenced resource focused on whether an earlier abstract claim is obsolete relative to the latest abstract of the same paper.

RevisionBench was intended to target exactly that setting. The proposed primary task is *current-claim selection*: given an earlier claim, a later claim, and the latest abstract, choose which claim remains current in the latest version. This framing is relevant for literature assistants and scientific verification systems, which can easily repeat stale metrics, outdated comparisons, or softened-away conclusions when papers have multiple versions.

The present workspace does not contain the adjudicated labels needed to complete that benchmark. The pipeline reached pilot export, but the required two-annotator plus adjudication workflow never happened. As a result, no honest benchmark evaluation, agreement analysis, or retained-item reporting can be given. We therefore position this paper explicitly as a reproducible failed-construction case study and resource audit, not as a benchmark-comparison paper.

Preprint. Under review.

That framing is the paper's actual deliverable. The contribution is not a new benchmark release, because `exp/build_benchmark/results.json` records `final_benchmark_size` $= 0$, `dev` $= 0$, `test` $= 0$, and `status` $=$ `"blocked_on_human_adjudication"`. The value of the current artifact is instead that it makes the construction failure reproducible and inspectable: future builders can see exactly how far a deterministic mining pipeline got, what it produced, and which missing human step prevented a benchmark claim.

Our contributions are:

- We adapt the original proposal into a fully reproducible construction report for RevisionBench, keeping the task definition but honestly narrowing the paper to the stages that were actually executed.
- We report exact construction statistics from the workspace: 135 in-scope papers retained from 180 raw records, 833 triggered candidates, 28 semantic-audit candidates, 1,428 rejected candidates, and 118 pilot items exported for each seed.
- We provide faithful artifact accounting for every experiment stage in `exp/`, including the negative result that the final adjudicated benchmark size is 0, agreement statistics are unavailable, and all retrieval, NLI, LLM, and prompt experiments were correctly skipped.

Section 2 reviews adjacent work, Section 3 describes the construction pipeline, Section 4 reports the executed artifacts, Section 5 summarizes reproducibility and artifact status, and Section 6 discusses limitations.

2 Related Work

Scientific revisions. Jiang et al. [2022] show that scientific papers can be aligned across versions with sentence-level annotations, while Jourdan et al. [2024] scale scientific revision alignment to a substantially larger corpus. These resources weaken any novelty claim based on revision histories themselves. RevisionBench differs only at the task level: it aims to isolate revision pairs that could support obsolete-versus-current claim discrimination under strictly local abstract evidence.

Revision-derived verification. VitaminC demonstrates that revisions can be turned into contrastive verification data [Schuster et al., 2021]. That precedent means RevisionBench cannot claim novelty for using revisions as supervision. The narrower distinction is the scientific-domain, abstract-local setting, where the evidence is intentionally restricted to the latest abstract.

Benchmark construction methodology. Recent benchmark papers also emphasize that credible resources depend on more than raw item volume. LiveBench foregrounds contamination control and continuous refresh as core design constraints rather than post hoc details [White et al., 2024]. ELAIPBench describes an explicitly role-separated annotation and verification workflow to make difficult academic-paper questions auditable [Dai et al., 2025]. Relative to this line of work, the main weakness of the current RevisionBench workspace is clear: it has a deterministic mining pipeline, but it lacks the completed human validation layer that turns mined candidates into a trustworthy benchmark.

Evolving knowledge and temporal verification. EvoWiki, EvolveBench, HoH, ChronoFact, and evolveQA all study changing facts, stale retrieval, or temporal verification [Tang et al., 2025, Ouyang et al., 2025, Zhu et al., 2025, Barik et al., 2025, Nakshatri et al., 2025]. These papers are the closest neighbors in spirit, but they focus on encyclopedia-style world knowledge, external evidence, or timeline reasoning rather than authored claim revisions within one paper's version chain.

Scientific claim verification. The closest adjacent benchmarks come from scientific claim verification. SciFact provides a benchmark for scientific claim verification with cited abstracts as evidence [Wadden et al., 2020]. MultiVerS strengthens that line with full-document evidence and weak supervision [Wadden et al., 2022]. SciVer extends the setting to multimodal scientific claim verification [Wang et al., 2025]. These resources matter for positioning: RevisionBench is not novel because it is "scientific" or because it is "claim verification." Its narrower intended contribution is version-conditioned stale-versus-current claim discrimination under latest-abstract-only evidence.

Unlike prior benchmark papers, the present submission does not report a finished evaluation resource. It reports a construction attempt that produced candidate pools and pilot packets but did not complete the human validation required to turn those candidates into a benchmark. In that sense, the paper is closer to a negative-result or lessons-learned construction note than to a benchmark release: its value lies in transparent pipeline and artifact reporting, not in a claim of benchmark difficulty or benchmark completion.

3 Method

3.1 Target Task and Scope

The intended primary task is *current-claim selection*: given an earlier claim c_e, a later claim c_l, and the latest abstract $a^\star$, select which claim remains current in $a^\star$. The benchmark is abstract-local by design: full-paper evidence is excluded to keep annotation criteria reproducible and tightly scoped.

In this workspace, no adjudicated labels were produced, so the paper reports only the construction pipeline up to pilot export. We therefore describe the intended task only to define the target artifact; we do not claim to have executed benchmark evaluation.

3.2 Source Collection

The source pool is collected from arXiv papers with at least two abstract versions. The implemented pipeline queries four CS areas: `cs.AI`, `cs.CL`, `cs.CV`, and `cs.LG`, with first-version submission dates between 2022-01-01 and 2025-12-31. Each record stores the arXiv ID, title, category metadata, and the abstract text for every observed version.

Filtering is deterministic. A record is retained only if it is in scope by category, has a valid monotonic revision chain, passes a lightweight English heuristic for every abstract version, and exhibits at least one adjacent-version abstract edit whose normalized edit distance exceeds 0.08. This stage retains 135 of 180 raw records.

3.3 Sentence Alignment and Candidate Mining

For each adjacent revision pair, the pipeline splits both abstracts into sentences and aligns sentence pairs with a lexical first pass. The lexical score combines token-set ratio, Levenshtein ratio, and content-word overlap:

$$s_{\text{lex}}(x, y) = 0.5\, s_{\text{set}}(x, y) + 0.3\, s_{\text{lev}}(x, y) + 0.2\, s_{\text{overlap}}(x, y).$$

Greedy one-to-one matches are kept only when $s_{\text{lex}} \geq 0.35$.

For each aligned pair (x, y), the pipeline computes sentence embeddings with `all-mpnet-base-v2` and measures cosine similarity. A pair is rejected as non-benchmark material if it is identical after normalization, differs only in punctuation, is too short, is a near-duplicate with cosine similarity above 0.985 and no numerical change, or is too weakly aligned under both lexical and embedding similarity.

Surviving pairs are assigned a provisional revision type using deterministic cues:

- `numeric_update` if the pair changes at least one number,
- `comparative_update` if it contains explicit comparative terms such as "better than" or "state-of-the-art",
- `softening_or_retraction` if it changes hedge or certainty language,
- `scope_qualification_change` if it changes scope qualifiers such as "all" or "zero-shot",
- `dataset_or_setting_change` otherwise.

The first two types form the coarse bucket `local_updates`; the remaining three form `semantic_updates`.

3.4 Triggered Pool and Semantic Audit

The pipeline produces two candidate pools. The main *triggered* pool keeps aligned pairs that satisfy the deterministic trigger logic above. For each triggered pair, the code additionally computes support proxies by scoring the earlier and later claims against the latest abstract sentence set with sentence-embedding similarity. These support scores are logged only as mining aids; they are not treated as gold labels.

The second pool is a *semantic audit* designed to identify semantically related edits that lexical alignment may underrepresent. A pair enters this pool if its lexical score is below 0.55 while cosine similarity is at least 0.78. This pool is not automatically benchmark-quality evidence. It is an audit slice intended for manual review, and the broader paper claims should depend on adjudicated retention rather than raw audit counts.

3.5 Pilot Export and Missing Adjudication

The intended annotation protocol requires two annotators plus adjudication. The code exports stratified pilot packets with up to 90 triggered candidates and up to 30 semantic-audit candidates per seed. In the current workspace, each of the three seeds exports 118 pilot items because only 28 audit candidates exist. However, the adjudicated annotation file is absent. The build stage therefore records benchmark construction as blocked, leaves the final benchmark size at 0, and produces no agreement, retained-item, or split-level benchmark statistics beyond zero-valued placeholders.

4 Experiments

4.1 Setup

All reported numbers come from executed artifacts in `exp/`. The environment uses Python 3.12.7 on Linux with one NVIDIA RTX A6000 GPU with 49,140 MiB reported memory, `torch` 2.10.0, `transformers` 4.44.2, `sentence-transformers` 5.3.0, `spaCy` 3.8.11, and fixed seeds 17, 23, and 42. The original plan requested Python 3.11, but that interpreter was unavailable on this machine, so environment compliance is recorded as partial.

Because no adjudicated benchmark exists, all downstream baselines were skipped: `LaterClaimWins`, retrieval, NLI, both LLM baselines, the main experiment, and prompt ablations all terminate with the same missing-benchmark condition. The experiments section therefore evaluates the construction pipeline itself and explicitly records what evidence is still missing.

4.2 Planned Versus Executed Study

The original proposal and experiment plan targeted a compact benchmark paper with a completed pilot annotation workflow, adjudicated obsolete/current labels, Wilson intervals for benchmark yield, frozen dev/test splits, and baseline evaluations over heuristic, NLI, and open-LLM systems. None of those benchmark-defining outputs exist in the current workspace. Instead, the executed artifacts stop at source collection, candidate mining, pilot export, and aggregate bookkeeping.

Table 3 makes that gap explicit. This contrast is important for interpretation: the current run supports claims about *candidate inventory and pipeline status*, but it does not support claims about annotation agreement, retained-item statistics, benchmark quality, or model difficulty.

4.3 Source Statistics

Table 1 summarizes the source pool. The pipeline starts from 180 raw multi-version records and retains 135 in-scope papers after filtering, a 75.0% keep rate. The only reported filter rejection reason is the absence of an adjacent abstract edit above threshold, which affects 45 papers. Among retained papers, the mean number of versions is 2.90 and the median is 2.0, while the latest abstract has a mean of 8.21 sentences and a median of 8.0. Every retained paper has at least one final-revision abstract edit, and 99.26% contain a numeric, comparative, or hedge-related edit somewhere in the chain.

Table 1: Source-pool statistics for the preliminary RevisionBench construction note. Percentages are computed from the executed data-collection artifact.

Statistic	Value
Raw multi-version records	180
Retained in-scope records	135
Keep rate	75.0%
Rejected for no adjacent edit above threshold	45
Mean versions per paper	2.90
Median versions per paper	2.0
Mean latest-abstract sentence count	8.21
Median latest-abstract sentence count	8.0
Papers with any final-revision abstract edit	100.0%
Papers with numeric/comparative/hedge edits	99.26%

Table 2: Construction-flow statistics for the executed workspace run. The benchmark remains incomplete because the adjudicated annotation file is missing.

Stage	Count
In-scope papers	135
Triggered candidates	833
Semantic-audit candidates	28
Rejected candidates	1,428
Pilot export size per seed	118
Final adjudicated benchmark size	0
Dev split size	0
Test split size	0

The retained source mix is 45 papers from `cs.CL`, 39 from `cs.CV`, 38 from `cs.LG`, and 13 from `cs.AI`. These counts show moderate topical spread, but they are still only source-pool statistics, not benchmark split statistics.

4.4 Candidate-Mining Results

Table 2 and Figure 1 summarize the construction flow. The mining stage yields 833 triggered candidate pairs, 28 semantic-audit candidates, and 1,428 rejected candidates. The primary pilot export contains 118 items for each seed, not the planned 120, because the semantic-audit pool is smaller than the target audit quota. The final adjudicated benchmark size remains 0, so both dev and test splits are 0.

Revision-type counts are shown in Table 4. The largest provisional category is `dataset_or_setting_change` with 283 items, followed by `softening_or_retraction` with 237, `scope_qualification_change` with 160, `numeric_update` with 103, and `comparative_update` with 50. These counts show that the triggered pool does not collapse purely to numeric edits, but they remain provisional because no human adjudication confirmed substantiveness or obsolete/current status.

4.5 Mining Ablation and Negative Findings

The planned ablations mostly depend on adjudicated labels and therefore could not run honestly. The mining ablation is the one partial exception. It records 30 sampled untriggered items and confirms that the semantic-audit pool is small: only 28 candidates satisfy the low-lexical and high-cosine criterion, which is 3.36% of the size of the triggered pool. The artifact also records `semantic_audit_exclusive_count` $= 0$. Because no adjudicated retention step exists, we describe this faithfully as zero semantic-audit-exclusive candidates rather than as zero retained benchmark items.

Table 3: Contrast between the originally planned benchmark paper and the executed workspace artifact. “Missing” means the corresponding result is absent rather than negative.

Component	Planned in proposal/plan	Executed result
Pilot annotation	120 candidates with two annotators plus adjudication	Pilot packets exported at 118 items per seed; no adjudicated pilot labels
Agreement reporting	Raw agreement and Cohen’s kappa	Missing
Retained benchmark items	150–220 Task 1 items expected after full workflow	Final benchmark size 0
Benchmark splits	Frozen dev/test splits	Dev 0, test 0
Construction uncertainty	Wilson intervals for yield	Missing
Baseline results	Heuristic, retrieval, NLI, and LLM comparisons	All skipped because adjudicated benchmark file is missing
Runtime ledger beyond environment metadata	Full preprocessing/inference runtime accounting	Only environment and stage-status artifacts recorded

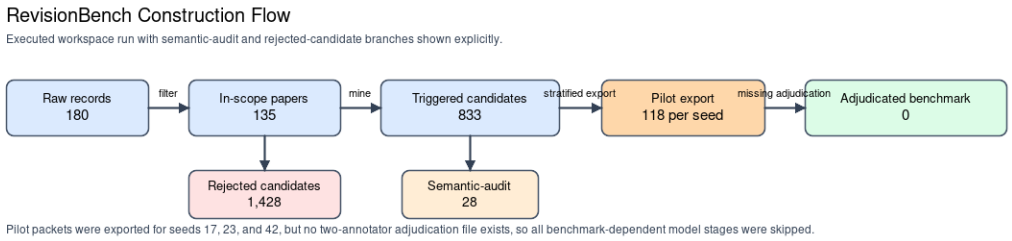

Figure 1: Faithful construction flow for the executed RevisionBench workspace. The figure includes the semantic-audit branch and rejected candidates that were omitted from the earlier draft. The pipeline reaches pilot export, but no two-annotator adjudication file exists, so the final benchmark size is 0.

This result should not be over-interpreted. Because there is no adjudicated benchmark, it does *not* prove that the semantic-audit mechanism is useless or that semantic updates are intrinsically rare. It only shows that, under the current source pool and thresholds, the audit path did not materially expand the candidate inventory before annotation.

4.6 Executed and Skipped Stages

Table 5 explicitly accounts for every experiment directory in `exp/`. Five stages executed and wrote substantive outputs: environment setup, data collection, candidate mining, mining ablation, and aggregate analysis. Ten additional stages ran only to record a blocked or skipped status: pilot silver export, build benchmark, evidence restriction, later-claim wins, retrieval, NLI, two LLM baselines, the main experiment, and prompt ablation. The key distinction is that the workspace contains real pipeline artifacts, but no stage that depended on adjudicated labels could proceed.

5 Reproducibility and Artifact Status

This project is reproducible in the narrow sense that every executed stage in `exp/` wrote machine-readable outputs and logs. The environment artifact records hardware, Python, package versions, and seeds. The aggregate analysis artifact records benchmark-flow counts and explicitly marks model analysis as `not_run`. The benchmark build artifact records the exact blocking condition: `label_source = none; adjudicated obsolete/current labels missing`.

The project is *not* reproducible as a finished benchmark release because the human annotation workflow is incomplete. The workspace contains no adjudicated benchmark file, no split manifest beyond the zero-sized placeholder, and no completed agreement statistics. This distinction matters: the correct reproducibility claim for this paper is that the *construction attempt* is reproducible, not that a benchmark has been reproduced.

Table 4: Provisional revision-type distribution in the 833 triggered candidates. These are mining labels, not adjudicated benchmark labels.

Revision type	Count	Share
Dataset or setting change	283	33.97%
Softening or retraction	237	28.45%
Scope qualification change	160	19.21%
Numeric update	103	12.36%
Comparative update	50	6.00%

Table 5: Artifact-level accounting for every stage under `exp/`. "Executed" means the stage produced substantive outputs, "Blocked" means the artifact records a hard prerequisite failure, and "Skipped" means the stage exited without attempting the benchmark-dependent computation.

Stage	Status	Evidence from results artifact
Environment setup	Executed	Python 3.12.7, RTX A6000, pinned package versions, seeds 17/23/42 recorded
Data collection	Executed	180 raw records, 135 retained in-scope papers, source statistics recorded
Candidate mining	Executed	833 triggered, 28 semantic-audit, 1,428 rejected candidates
Mining ablation	Executed	30 sampled untriggered items, 28 audit candidates, 0 semantic-audit-exclusive candidates
Aggregate analysis	Executed	Aggregate flow counts and subject-area counts recorded; model results empty
Pilot export	Skipped	`status` = `skipped`; 118 pilot items were exported per seed, but no adjudicated labels were produced
Benchmark build	Blocked	`final_benchmark_size` = 0 and `status` = `"blocked_on_human_adjudication"`
Evidence restriction ablation	Skipped	requires additional human relabeling with and without full-paper access
Later-claim-wins baseline	Skipped	missing final adjudicated benchmark file
Retrieval baseline	Skipped	missing final adjudicated benchmark file
NLI baseline	Skipped	missing final adjudicated benchmark file
Qwen2.5-7B-Instruct	Skipped	missing final adjudicated benchmark file
Llama-3.1-8B-Instruct	Skipped	missing final adjudicated benchmark file
Main experiment	Skipped	missing final adjudicated benchmark file
Prompt ablation	Skipped	missing final adjudicated benchmark file

6 Discussion and Limitations

The main lesson from this study is operational. RevisionBench's core bottleneck is not candidate generation but trustworthy adjudication. The mining pipeline can readily produce hundreds of candidate claim updates from a modest source pool, but none of those counts by themselves justify a benchmark claim. Without adjudicated obsolete/current labels, the project remains a pilot-ready construction audit rather than a benchmark release.

The candidate statistics are still useful. First, the pipeline shows that abstract revisions are common in the targeted CS slice: all 135 retained papers contain at least one final-revision abstract edit. Second, the provisional distribution suggests that interesting semantic phenomena are present, especially softening, scope qualification, and dataset-setting changes. Third, the semantic-audit path is much smaller than expected in this run, which is a useful negative result for future benchmark builders who may otherwise rely too heavily on paraphrase recovery to populate rare categories.

This study has several limitations. The source pool is only 135 retained papers, below the original proposal target. The revision taxonomy is fully automatic at this stage, so category counts are descriptive rather than validated. No annotation agreement, obsolete rate, retained-item rate, adjudication rate, or benchmark split statistics can be reported because the annotation workflow never completed. Finally,

all model baselines and prompt comparisons remain untested here, so the paper cannot support any claims about benchmark difficulty for NLI systems or LLMs.

The immediate next step is straightforward but labor-intensive: complete the planned two-annotator pilot and adjudication pass over the exported 118-item packets, freeze the guidelines, and rebuild the benchmark from human labels rather than mining heuristics.

7 Conclusion

RevisionBench targets a narrow problem: identifying when an earlier scientific abstract claim has become obsolete relative to the latest abstract of the same paper. In this workspace, the implemented pipeline successfully reaches a pilot-ready construction stage, retaining 135 in-scope papers from 180 raw records and mining 833 triggered candidate pairs plus 28 semantic-audit candidates. The decisive result, however, is that no benchmark was produced: the final adjudicated benchmark size remains 0 because the planned annotation and adjudication workflow was never completed.

The paper should therefore be read as a reproducible failed-construction case study rather than as a benchmark release. Its contribution is to make the failure legible and falsifiable: the workspace shows exactly which source pool was assembled, how the candidate inventory was generated, which pilot packets were exported, and where the process became blocked on missing human adjudication.

That negative outcome still matters. It shows that a reproducible candidate-mining pipeline is feasible, but that benchmark validity depends on human adjudication rather than mining volume alone. The appropriate claim for the current artifact is therefore modest: this is a transparent construction note and resource audit about an unfinished benchmark, not a benchmark paper.

References

Anab Maulana Barik, Wynne Hsu, and Mong Li Lee. Chronofact: Timeline-based temporal fact verification. In *IJCAI 2025*, 2025. URL `https://www.ijcai.org/proceedings/2025/893`.

Xinbang Dai, Huikang Hu, Yongrui Chen, Jiaqi Li, Rihui Jin, Yuyang Zhang, Xiaoguang Li, Lifeng Shang, and Guilin Qi. Elaipbench: A benchmark for expert-level artificial intelligence paper understanding. *arXiv preprint arXiv:2510.10549*, 2025. URL `https://arxiv.org/abs/2510.10549`.

Chao Jiang, Wei Xu, and Samuel Stevens. arxivedits: Understanding the human revision process in scientific writing. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 9420–9435. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.emnlp-main.641. URL `https://aclanthology.org/2022.emnlp-main.641/`.

Léane Jourdan, Florian Boudin, Nicolas Hernandez, and Richard Dufour. Casimir: A corpus of scientific articles enhanced with multiple author-integrated revisions. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, pages 2883–2892. ELRA and ICCL, 2024. URL `https://aclanthology.org/2024.lrec-main.257/`.

Nishanth Sridhar Nakshatri, Shamik Roy, Manoj Ghuhan Arivazhagan, Hanhan Zhou, Vinayshekhar Bannihatti Kumar, and Rashmi Gangadharaiah. When facts change: Probing llms on evolving knowledge with evolveqa. *arXiv preprint arXiv:2510.19172*, 2025. URL `https://arxiv.org/abs/2510.19172`.

Jie Ouyang, Tingyue Pan, Mingyue Cheng, Ruiran Yan, Yucong Luo, Jiaying Lin, and Qi Liu. Hoh: A dynamic benchmark for evaluating the impact of outdated information on retrieval-augmented generation. In *ACL 2025*, 2025. URL `https://aclanthology.org/2025.acl-long.301/`.

Tal Schuster, Adam Fisch, and Regina Barzilay. Get your vitamin c! robust fact verification with contrastive evidence. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 624–643. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.naacl-main.52. URL `https://aclanthology.org/2021.naacl-main.52/`.

Wei Tang, Yixin Cao, Yang Deng, Jiahao Ying, Bo Wang, Yizhe Yang, Yuyue Zhao, Qi Zhang, Xuanjing Huang, Yu-Gang Jiang, and Yong Liao. Evowiki: Evaluating llms on evolving knowledge. In *ACL 2025*, 2025. URL https://aclanthology.org/2025.acl-long.47/.

David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7534–7550. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.609. URL https://aclanthology.org/2020.emnlp-main.609/.

David Wadden, Kyle Lo, Lucy Lu Wang, Arman Cohan, Iz Beltagy, and Hannaneh Hajishirzi. Multivers: Improving scientific claim verification with weak supervision and full-document context. In *Findings of the Association for Computational Linguistics: NAACL 2022*, pages 61–76. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.findings-naacl.6. URL https://aclanthology.org/2022.findings-naacl.6/.

Chengye Wang, Yifei Shen, Zexi Kuang, Arman Cohan, and Yilun Zhao. Sciver: Evaluating foundation models for multimodal scientific claim verification. In *ACL 2025*, 2025. URL https://aclanthology.org/2025.acl-long.420/.

Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, Shubh Agrawal, Sandeep Singh Sandha, Siddartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. Livebench: A challenging, contamination-limited LLM benchmark. *arXiv preprint arXiv:2406.19314*, 2024. URL https://arxiv.org/abs/2406.19314.

Zhiyuan Zhu, Yusheng Liao, Zhe Chen, Yuhao Wang, Yunfeng Guan, Yanfeng Wang, and Yu Wang. Evolvebench: A comprehensive benchmark for assessing temporal awareness in llms on evolving knowledge. In *ACL 2025*, 2025. URL https://aclanthology.org/2025.acl-long.788/.

A Annotation Materials

The intended pilot annotation workflow is preserved here so the missing human stage is explicit rather than implicit.

Annotator inputs. Each item shows the earlier sentence, later sentence, earlier abstract, latest abstract, paper identifier, and version identifiers.

Stage A labels. Annotators choose one of `substantive_change`, `non_substantive_edit`, or `ambiguous`.

Stage B labels. For substantive items, annotators label the earlier claim against the latest abstract as `obsolete`, `supported_current`, or `insufficient_evidence`. Annotators also mark the supporting latest-abstract span and the final revision type.

Inclusion rule. An item enters the intended primary benchmark only if adjudication confirms a substantive change, the earlier claim is obsolete, and the later claim is the current counterpart under latest-abstract evidence.

B Artifact Inventory

C Skipped-Stage Rationale

The skipped stages fall into two groups. The `pilot_silver` and `build_benchmark` stages are blocked by missing two-annotator adjudication. The `evidence_restriction` ablation has a different blocker: it requires fresh human relabeling with and without full-paper access, which no annotator pool or adjudicator in this workspace can honestly supply. All later modeling stages then fail for the same concrete reason: the file `data/final/adjudicated_benchmark.jsonl` does not exist.

Table 6: Files and artifacts needed to reproduce the current construction note. All entries refer to artifacts present in the workspace rather than hypothetical future benchmark files.

Artifact	Purpose
`proposal.md`	Original introduction, task framing, related work, and benchmark proposal
`plan.json`	Experiment plan, budget assumptions, and preregistered stage descriptions
`exp/environment_setup/results.json`	Hardware, software, and seed ledger
`exp/data_collection/results.json`	Raw and filtered source-pool statistics
`exp/candidate_mining/results.json`	Triggered, audit, rejected, and provisional type counts
`exp/ablation_mining/results.json`	Semantic-audit negative finding and untriggered sample count
`exp/build_benchmark/results.json`	Benchmark-blocking status and zero-sized splits
`exp/analysis/results.json`	Aggregate flow counts, subject-area counts, and empty model ledger
`exp/*/SKIPPED.md`	Human-readable reasons for skipped stages
`references.bib` and `references/`	Bibliography and parsed reference papers